A priori power ca cu ations using simu ation

Effect size, sample size and significance level

Effect size

0 2

0 0.8 1.6

Sample size

0 100

1 41 81

Significance level (alpha)

0 1

0 0.4 0.8

Distribution of p-values computed from a two-sample t-test, for a given effect size (Cohen's d), sample size and significance level.

Disconnected from the server.
Reload

Statistical power

		Null hypothesis is false	Null hypothesis is true
Decision	Reject null hypothesis	Correct decision ($p = 1 - \beta$)	Type I error ($p = \alpha$)
	Do not reject null hypothesis	Type II error ($p = \beta$)	Correct decision ($p = 1 - \alpha$)

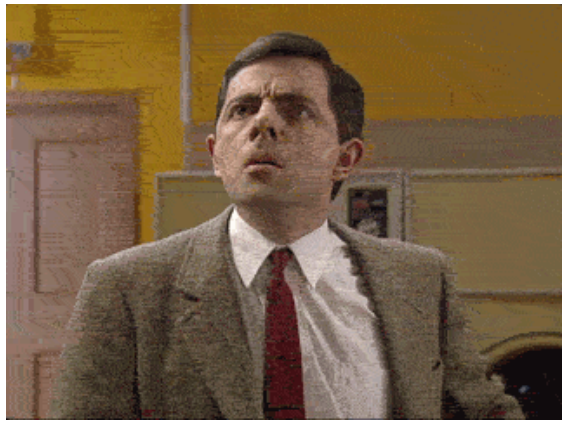
Statistical power, more broadly

Kruschke, 2015

- Obtaining a p-value inferior to α (where α is your favourite significance level)
- Obtaining a Bayes Factor (or a Likelihood Ratio) superior to some threshold T
- Rejecting a null value of a parameter (e.g., by showing that a region of practical equivalence (ROPE) around the null value excludes the 95% CI/CrI)
- Affirming a predicted value of a parameter (e.g., by showing that a ROPE around the predicted value includes the 95% CI/CrI)
- Achieving some level of precision in the estimate of a parameter (e.g., the width of the 95% CI/CrI)

Why simulation ?

G*Power



General steps to conduct power analyses via simulation

Kruschke, 2015

1. (Generate representative parameter values from hypothesis)
2. Generate a random sample of data from the hypothetical parameter values
3. Retrieve the (posterior distribution of the) estimate of interest based on the fake dataset
4. Tally whether goal was attained
5. Repeat many (many) times

A gentle introduction to power analysis via simulation

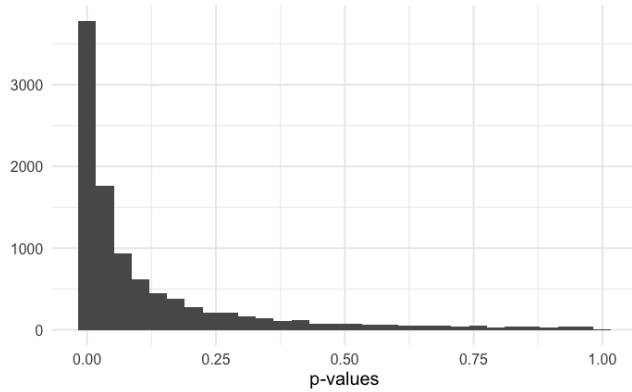
```r
library(tidyverse)

sim_power <- function(n, d) {

    x <- rnorm(n, mean = 0, sd = 1)
    y <- rnorm(n, mean = d, sd = 1)
    p <- t.test(x, y)$p.value

    return(p)

}
```

```r
sim_power(n = 36, d = 0.5)
```

```
[1] 0.003069109
```

A gentle introduction to power analysis via simulation

```
# replicating the sim_power function 1e4 times
results <- replicate(1e4, sim_power(n = 36, d = 0.5) )

# plotting it
results %>% qplot + theme_minimal(base_size = 20) + xlab("p-values")
```




```
# what proportion of p-values are below .05 ?
sum(results < .05) / length(results)
```

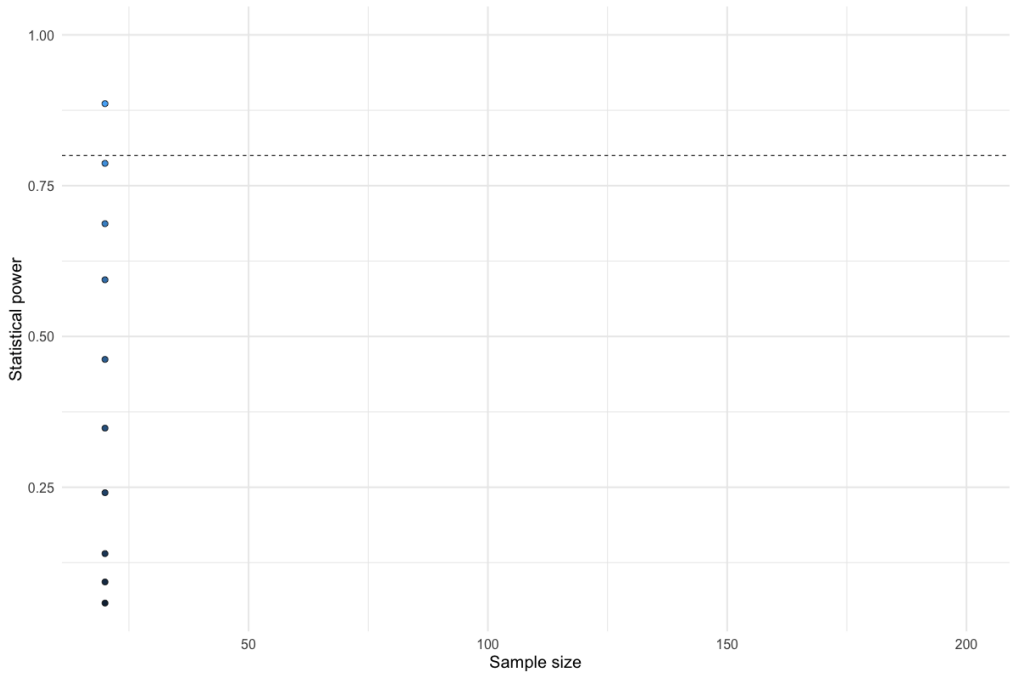
```
[1] 0.5476
```

Aparté: the distribution of p-values

A gentle introduction to power analysis via simulation

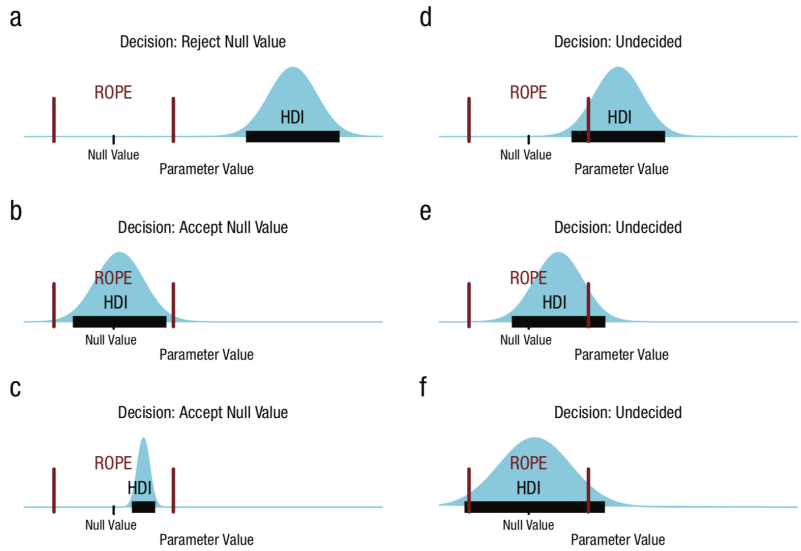
```
# set up parameters
crossing(n = seq(20, 200, 10), d = seq(0.1, 1, 0.1) ) %>%
    # for each sample size and effect size
    group_by(n, d) %>%
    # add observations
    expand(reps = 1:n) %>%
    ungroup %>%
    # add 1000 simulations
    crossing(sim = 1:1e3) %>%
    # generate data
    mutate(x = rnorm(n(), mean = 0, sd = 1), y = rnorm(n(), mean = d, sd = 1) ) %>%
    # for each sample size, effect size, and simulated dataset
    group_by(n, d, sim) %>%
    # compute a t-test and extract the p-value
    mutate(p = t.test(x, y, data = .)$p.value) %>%
    ungroup %>%
    # for each sample size and effect size
    group_by(n, d) %>%
    # compute power
    summarise(power = 1 - sum(p > 0.05) / n() ) %>%
    ungroup
```

A gentle introduction to power analysis via simulation



Another example using the ROPE + HDI procedure

Kruschke, 2018



Another example using the ROPE + HDI procedure

Another example using the ROPE + HDI procedure

```
sim_data <- function(n, beta12) {

    # set coefficients
    alpha <- 10
    beta1 <- 0.3
    beta2 <- - 0.5

    # generate design structure
    x1 <- c(rep(c(0), n / 2), rep(c(1), n / 2) )
    x2 <- rep(c(rep(c(0), n / 4), rep(c(1), n / 4) ), 2)
    sigma <- rnorm(n, 0, sd = 1) # random noise, with standard deviation of 1

    # generate data using the regression equation
    y <- alpha + beta1 * x1 + beta2 * x2 + beta12 * x1 * x2 + sigma

    # join the variables in a common dataframe
    data <- data.frame(cbind(x1, x2, y) )

    # ouput this dataframe
    return(data)

}
```

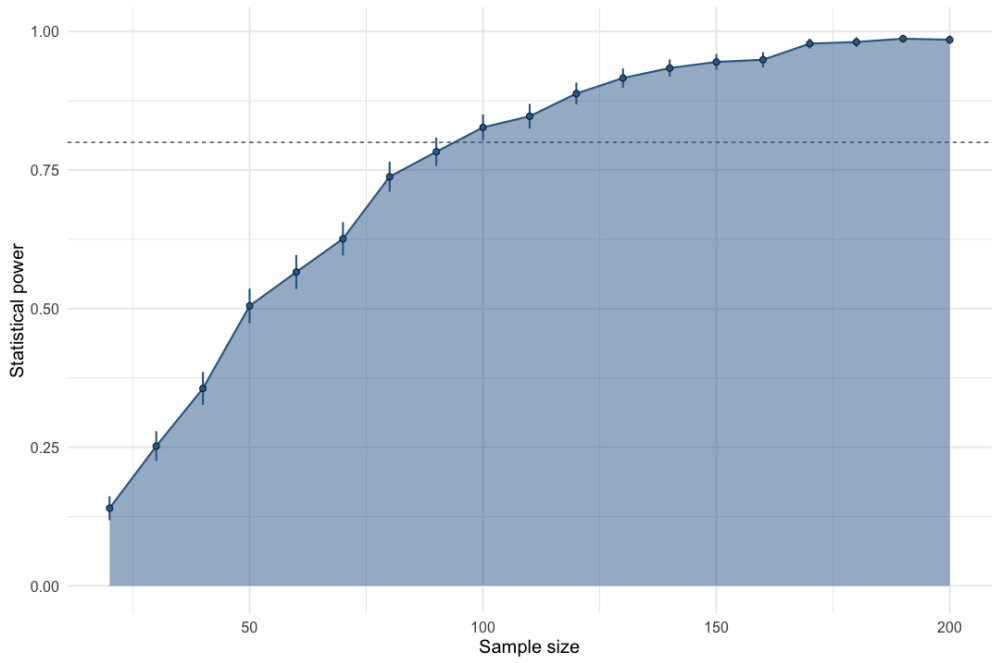
Another example using the ROPE + HDI procedure

```r
# defining the combinations of parameters values to explore
grid <- crossing(n = seq(20, 200, 10), sim = 1:1e3)

# hypothesised value for beta12
beta12 <- 1.1

# initialising the results dataframe
res <- data.frame(n = numeric(), beta12 = numeric(), lower = numeric(), upper = numeric() )

for (i in 1:nrow(grid) ) {

    # generating data
    df <- sim_data(n = grid$n[i], beta12 = beta12)

    # fitting the model
    BF <- BayesFactor::lmBF(
        y ~ 1 + x1 * x2, data = df,
        posterior = TRUE, iterations = 1e3, progress = FALSE
        )

    # retrieving the posterior samples
    samples <- BF[, 4]

    # computing the 95% credible interval
    CrI <- quantile(samples, probs = c(0.025, 0.975) )

    # storing the results
    res[i, ] <- cbind(grid$n[i], beta12, CrI[1], CrI[2])

}
```

Another example using the ROPE + HDI procedure

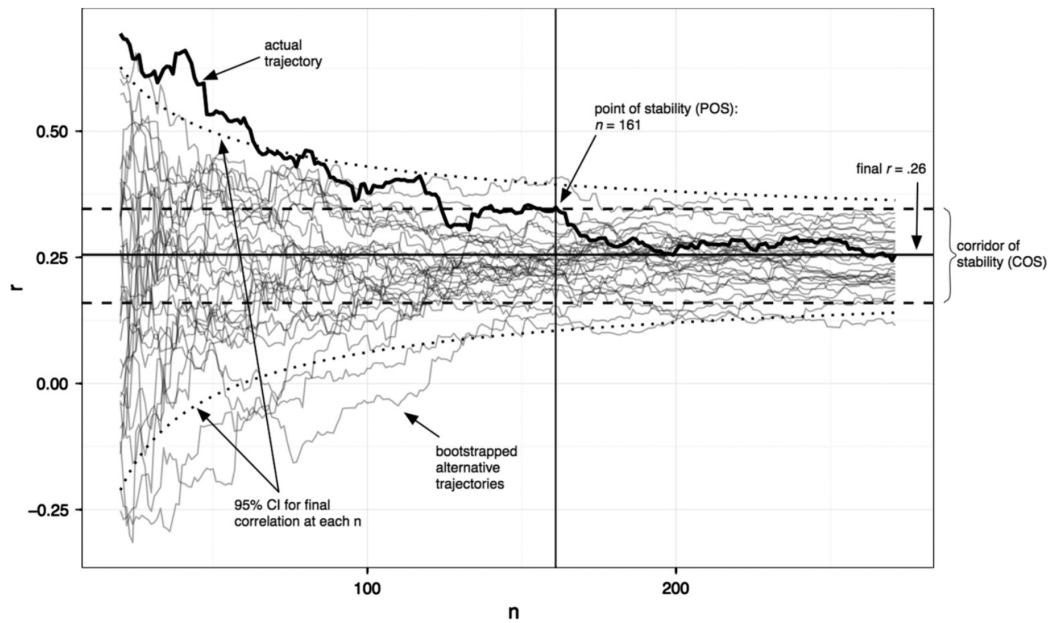
```
# defining the ROPE
rope <- c(-.1, .1)

# approximating the power function
res %>%
    mutate(
        success = case_when(
            lower < min(rope) & upper < min(rope) ~ 1,
            lower > min(rope) & upper < max(rope) ~ 1,
            lower > min(rope) & upper > max(rope) ~ 1,
            TRUE ~ 0
            )
        ) %>%
    group_by(n, beta12) %>%
    summarise(power = mean(success) ) %>%
    ungroup %>%
    # plotting it
    ggplot(aes(x = n, y = power, group = beta12, fill = beta12) ) +
    geom_hline(yintercept = 0.8, linetype = 2) +
    geom_area(alpha = 0.5) +
    geom_line(aes(colour = beta12), size = 1) +
    geom_point(shape = 21, size = 3) +
    xlab("Sample size") +
    ylab("Statistical power") +
    theme_minimal(base_size = 20) +
    theme(legend.position = "none")
```

Another example using the ROPE + HDI procedure



Planning for precision

Schönbrodt & Perugini, 2013



Planning for precision

Kruschke, 2015

basic statistics **paper**
package **shiny app**

Take-home messages, further resources

-
-
- generate estimate tally
  repeat

This blogpost
vignette blogpost
on power simulations blogpost
this one